SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1409 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I **S SUNDARESAN / 192411245** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i <= n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5}
   int total;

   total = sumArray(arr);

   if(total == 15) {
      printf("Correct Sum\n")
   }

   int a = 10, b = 0;
   int c = a / b;

   int *p;
   *p = 20;

   return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number

3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

ANSWER:

DESCRIPTION:

The provided code sample highlights the cases when syntax errors (missing semicolons) interrupt the compilation process and prevent subsequent semantic analysis. The code also demonstrates that logical errors such as array overflow, division by zero, and dereference of a null pointer are allowed through static compilation and cause program failure at runtime. It

becomes possible to resolve the identified issues by investigating their impact on the program logic and syntax and eliminating these issues.

1.Line 5: Array Out-of-Bounds Access
Code: for(i = 0; i <= n; i++)
Phase: Logical / Runtime Error

Description: This code will cause an array out-of-bounds memory access error. The loop should iterate while i < n since the array indices start at 0 and end at n – 1. Using the <= operator will result in accessing a memory address that does not belong to the array resulting in memory corruption or including garbage values in the sum.

2. Line 11: Missing Semicolon After Array Declaration
Code: int arr[5] = {1,2,3,4,5}
Phase: Syntax Error

Description: In C programming language syntax, each statement should end with a semicolon ; symbol. The absence of this character confuses the compiler and causes a syntax error since the boundary of the statement cannot be identified.

3. Line 14: Incorrect Number of Function Arguments
Code: total = sumArray(arr);
Phase: Semantic Error

Description: The function sumArray requires two arguments namely: the integer array and the number of elements in the array. However, the function is called with only one argument hence a semantic error will occur since the function signature does not match the definition.

4. Line 17: Missing Semicolon After printf Statement
Code: printf("Correct Sum\n")
Phase: Syntax Error

Description: Similar to the first error, this code causes a syntax error because the printf statement is not terminated with a semicolon ;.

5. Line 21: Division By Zero
Code: int a = 10, b = 0; int c = a / b;
Phase: Runtime Error

Description: This line will cause a division by zero error at run-time. The code attempts to execute 10 / 0 which is mathematically undefined. A floating point exception (SIGFPE) will be raised and the program execution aborted since the CPU cannot perform this computation.

6. Line 24: Dereferencing Uninitialized Pointer

Code: int p; p = 20;

Phase: Runtime Error

Description: This code tries to dereference a pointer variable that has not been initialized to point to a valid memory address. The assignment operation will result in writing the value 20 to a random garbage memory location hence a segmentation fault (SIGSEGV) error.

CORRECTED CODE:

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i < n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1, 2, 3, 4, 5};
   int total;
```

```
    total = sumArray(arr, 5);

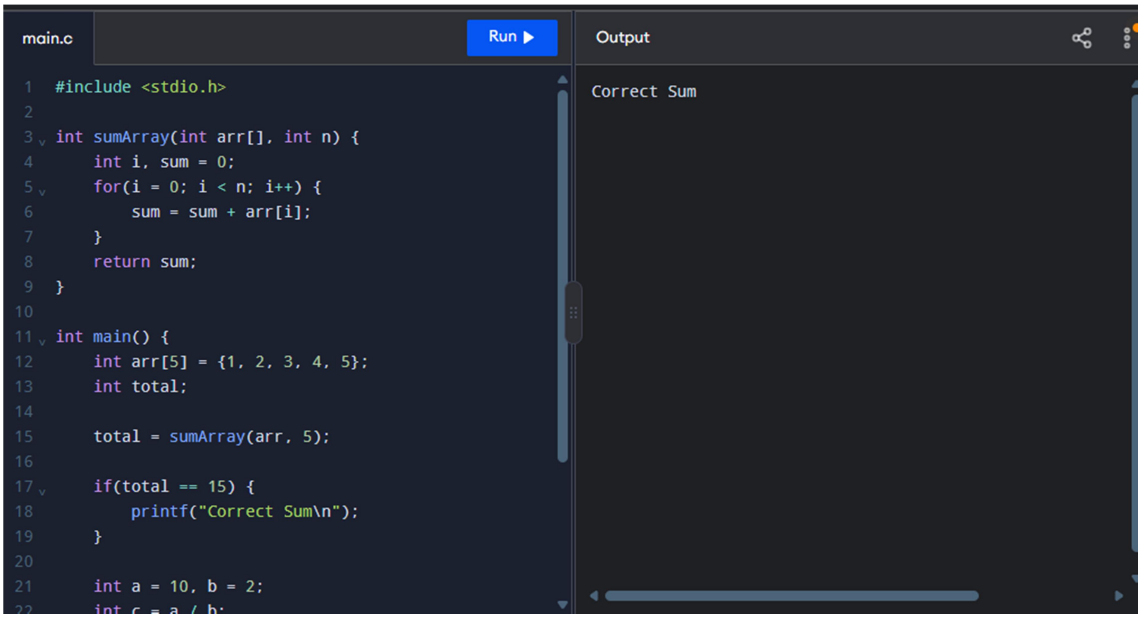
    if(total == 15) {
        printf("Correct Sum\n");
    }

    int a = 10, b = 2;
    int c = a / b;

    int val = 0;
    int *p = &val;
    *p = 20;

    return 0;
}
```

OUTPUT:



Q2.The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
    int 1num = 100;
    float rate@ = 25.5;
```

```
    char ch = 'abc';
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;

    printf("Welcome to Compiler Design);

    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;

    char str1[] = "Hello;
    char str2[] = "World";

    int _valid = 10;
    int invalid-id = 20;

    float value = 5..6;
    int data = 10@5;

    char c = 'xy';
    int 3value = 60;

    printf("Value: %d\n", total);

    return 0;
}
```

Tasks

(a) Error Identification

i) Identify all lexical errors in the program.

ii) List the line number and the invalid token.

(b) Error Classification

i) For each error, explain why it is considered a lexical error.

ii) Specify which rule of token formation is violated:

iii) Identifier rules

iv) Constant formation

v) String/character literal rules

vi) Operator/symbol rules

(c) Token Correction

i)Rewrite each invalid token into a valid token.

ii)Example: 2value → value2

(d) Compiler Behavior Analysis

i) Explain how a lexical analyzer (scanner) processes such errors.

ii)Discuss whether the compiler:

iii)Stops immediately

iv)Continues scanning using error recovery

(e) Advanced Thinking

i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.

ii) Suggest how error messages can be improved for beginners.

ANSWER:

DESCRIPTION:

This question concerns the Lexical Analysis (Scanning) stage of a compiler, by exploring how invalid tokens and incorrectly formed constructs are handled. By analyzing the rules for forming identifiers, numeric constants, character/strings literals and operators in the C language, the problem can be solved by answering the following question: how does a scanner process bad lexemes using Deterministic Finite Automata and error recovery mechanisms to skip erroneous tokens.

Line 4: int 1num = 100;

Invalid Identifier. Identifiers in C cannot begin with a numeric digit. Renaming it to something like num1 resolves the issue.

Line 5: float rate@ = 25.5;

Illegal Symbol in Identifier. Special characters like @ are not allowed in C identifier names (only letters, numbers, and underscores are valid).

Line 6: char ch = 'abc';

Malformed Character Literal. A character literal enclosed in single quotes ' ' can only store a single character, not multiple characters like 'abc'.

Line 7: int total = 50$20;

Invalid Operator / Illegal Symbol. The $ character is not a recognized operator in C for performing arithmetic operations.

Line 8: int num#1 = 30;

Illegal Character in Identifier. The # symbol cannot be part of a variable name in C; it is reserved for preprocessor directives.

Line 9: int %count = 40;

Syntax/Operator Violation. The % character is the modulus operator in C and cannot be placed at the start of a variable declaration name.

Line 11: printf("Welcome to Compiler Design);

Unterminated String Literal. The string literal is missing its closing double quotation mark " before the ending parenthesis and line boundary.

Line 13: int x = 09;

Invalid Octal Constant. Numbers starting with 0 are treated as base-8 (octal) constants in C, which only accept digits from 0 to 7. 9 is an invalid octal digit.

Line 14: int y = 0xZ12;

Invalid Hexadecimal Constant. Hexadecimal constants prefixed with 0x only allow digits 0-9 and letters A-F. The letter Z is an invalid hex character.

Line 15: int z = 12.3.4;

Malformed Floating-Point Constant. A numeric literal cannot contain more than one decimal point.

Line 17: char str1[] = "Hello;

Unterminated String Literal. The string literal is missing a closing double quote ", causing the semicolon to be treated as part of the string rather than a statement terminator.

Line 21: int invalid-id = 20;

Illegal Hyphen in Identifier. The hyphen - is interpreted by the compiler as a subtraction operator rather than a valid identifier character.

Line 23: float value = 5..6;

Malformed Floating Constant. Having two consecutive decimal points .. creates an invalid numeric token sequence.

Line 24: int data = 10@5;

Illegal Character in Expression. The @ symbol is an unrecognized lexeme that cannot be used as an operator between two numeric values.

Line 26: char c = 'xy';

Multi-Character Literal. Single quotes can hold only one character byte; passing two characters 'xy' violates character literal formatting rules.

Line 27: int 3value = 60;

Invalid Identifier. Identifiers cannot start with a digit (3), as it confuses the lexical analyzer between number tokens and identifier tokens.

CORRECTED CODE:

```
#include <stdio.h>

int main() {
    int num1 = 100;
    float rate_val = 25.5;
    char ch = 'a';
    int total = 50 + 20;
```

```
    int num_1 = 30;
    int count = 40;

    printf("Welcome to Compiler Design");

    int x = 9;
    int y = 0xA12;
    float z = 12.34;

    char str1[] = "Hello";
    char str2[] = "World";

    int _valid = 10;
    int invalid_id = 20;

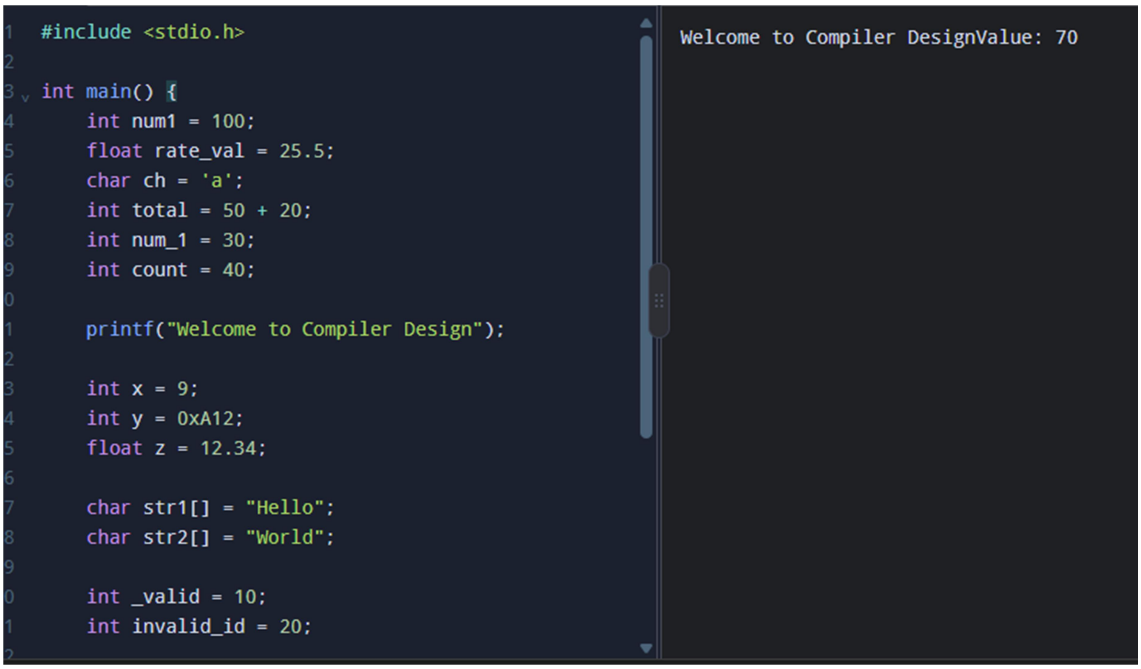
    float value = 5.6;
    int data = 10 + 5;

    char c = 'x';
    int value3 = 60;

    printf("Value: %d\n", total);

    return 0;
}
```

OUTPUT:



Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

A. Lexical Errors
B. Syntax Errors
C. Semantic Errors
D. Intermediate Code Generation Issues (logic/precedence)
E. Code Optimization Issues (redundancy/inefficiency)
F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;

   int arr[4] = {1,2,3,4};
   printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;

   int total;
   total = x + y * result + 0;

   while(x < 20) {
```

```
        printf("%d\n", x);
    }

    return 0;
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

I. Identify **all errors** in the program.
II. Mention **line number + error description**.

(b) Classify by Compiler Phase

Classify each error into:

I. Lexical (invalid symbols like @num)
II. Syntax (missing semicolon, brackets)
III. Semantic (function argument mismatch)
IV. Intermediate Code Issues (wrong expression handling/precedence)
V. Optimization Issues (redundant + 0)
VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

I. Which errors are detected at **compile time**?
II. Which errors appear only during **execution**?
III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application)

I. Rewrite the **fully corrected program**.
II. Ensure:
    A. Proper syntax
    B. Correct semantics
    C. Safe execution

(e) Deep Analysis (Evaluation)

I. Explain how **incorrect pointer usage** affects runtime.
II. Analyze how **array bounds violation** impacts memory.
III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

I. Suggest **2 improvements** in compiler design to:
    A. Detect runtime errors earlier
    B. Improve debugging messages

II. Propose how **optimization phase** can remove redundant expressions.

DESCRIPTION:

This question seeks to explore the various phases of the program compilation process and their connection with errors. The errors include lexical, syntax, and semantic ones, with the last one comprising type checking and code generation for expression and control structures, intermediate code optimization, and execution. It wants to see how the compiler handles errors at different stages and whether a successfully compiled program can still have errors at later stages. For example, a division by zero, illegal indirect reference, array subscript out of bounds, and infinite loop will be handled during execution, while other errors are discovered too late in the compilation sequence to be eliminated by the compiler.

Line 4: `return a / b`

Description: Missing Semicolon. The statement is missing a terminating semicolon `;`, which will cause a syntax error during syntax analysis.

Line 8: `int #value = 20;`

Description: Illegal Character in Identifier. The `#` symbol is not a valid token character for identifiers in C and is rejected by the lexical analyzer.

Line 9: `float num = 15.5`

Description: Missing Semicolon. The variable declaration is missing a terminating semicolon `;`.

Line 12: `result = divide(10);`

Description: Function Argument Mismatch. The function `divide` expects two arguments (`a` and `b`), but only one was passed, which fails the semantic signature checking.

Line 14: `if(result = 5)`

Description: Assignment Used Instead of Equality. The assignment operator `=` is used instead of the relational operators `==`. This would continuously assign `5` into `result` and evaluate to true, changing the program control flow.

Line 15: `printf("Result is 5\n")`

Description: Missing Semicolon. The `printf` statement is missing a terminating semicolon `;`.

Line 20: `int z = x / y;`

Description: Division by Zero. Variable `y` evaluates to `0`. Integer division by zero is undefined in C and raises a hardware arithmetic exception (`SIGFPE`) at run-time.

Line 23: `arr[4]`

Description: Array Index Out of Bounds. An array declared with size `4` has valid indices ranging from `0` to `3`. Trying to access index `4` will read either unallocated or adjacent stack memory.

Line 26: `ptr = 25;`

Description: Uninitialized Pointer Dereference. The pointer `ptr` is declared, but not initialized with a valid memory address and will raise a Segmentation Fault (`SIGSEGV`) when dereferenced.

Line 29: `total = x + y result + 0;`

Description: Redundant Expression / Inefficiency. The addition of `+ 0` serves no purpose and is a redundant expression that can be optimized away by the compiler.

Line 31–33: `while(x < 20)`

Description: Infinite Loop / Unreachable Termination. The loop control variable `x` is never updated or incremented inside the loop body and will result in an infinite loop.

CORRECTED CODE:

```
#include <stdio.h>

int divide(int a, int b) {
   if (b == 0) return 0;
   return a / b;
}

int main() {
   int value = 20;
   float num = 15.5;
```

```
    int result;

    result = divide(10, 2);

    if(result == 5) {
        printf("Result is 5\n");
    }

    int x = 10;
    int y = 2;
    int z = x / y;

    int arr[4] = {1, 2, 3, 4};
    printf("%d\n", arr[3]);

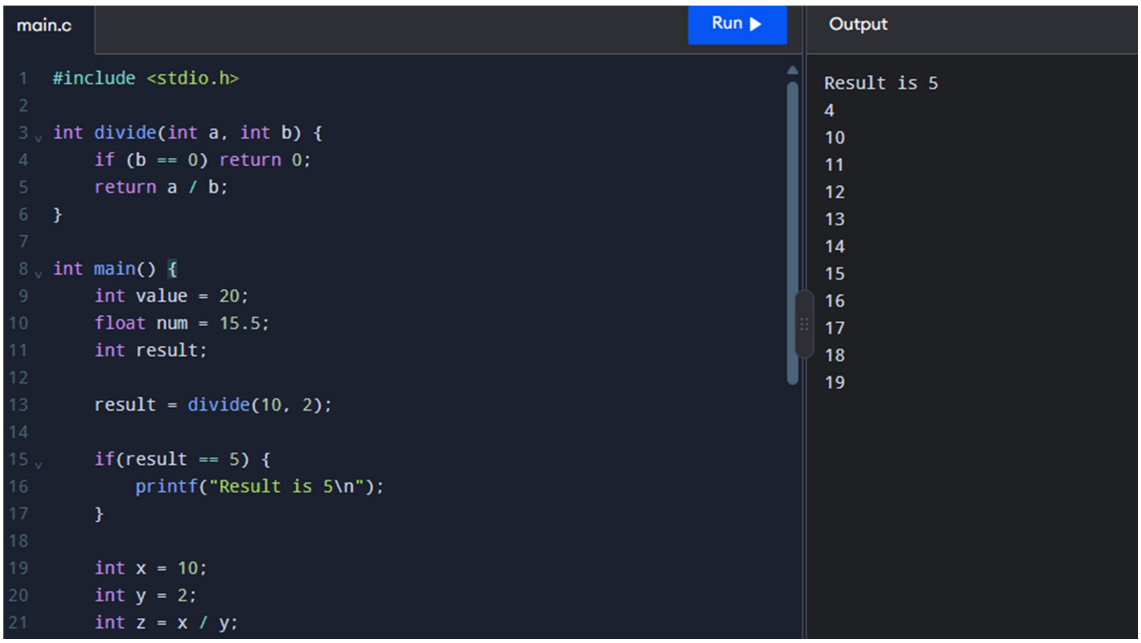
    int temp = 0;
    int *ptr = &temp;
    *ptr = 25;

    int total;
    total = x + y * result;

    while(x < 20) {
        printf("%d\n", x);
        x++;
    }

    return 0;
}
```

OUTPUT:


main.c
Run
Output
#include <stdio.h>
int divide(int a, int b) {
if (b == 0) return 0;
return a / b;
}
int main() {
int value = 20;
float num = 15.5;
int result;
result = divide(10, 2);
if(result == 5) {
printf("Result is 5\n");
}
int x = 10;
int y = 2;
int z = x / y;
Result is 5
4
10
11
12
13
14
15
16
17
18
19

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand